Assessment: E-commerce Returns & Refund Processing

Technology: Microsoft Agent Framework, Azure AI Foundry, Python
Duration: 3 Hours
Topics: Agent Workflow Design, Task Delegation, Conditional Routing, Human-in-the-Loop

Business Scenario

An e-commerce company receives customer requests for product returns, refunds, and replacements.

The company wants to automate the initial review using multiple AI agents while ensuring that high-value, risky, or exceptional requests receive human approval.

Build a graph-based multi-agent application using Microsoft Agent Framework and Azure AI Foundry.

Task 1 — Read and Process Customer Requests

Use the supplied `returns.json` file containing 10 customer requests.

Each request contains:

- Request ID and order ID
- Product and category
- Refund or replacement request
- Product amount
- Days since delivery
- Item condition
- Return reason
- Invoice and photo evidence availability
- Previous returns within 90 days

Load the records dynamically into Python.

Task 2 — Create Specialized AI Agents

Create the following agents:

Agent 1: Eligibility Agent

Review each return request against the company's return policy. Identify whether the request meets eligibility requirements and explain any exceptions.

Agent 2: Risk Agent

Analyze potential risks such as unusual return frequency, missing evidence, or high-value requests. Provide a short risk assessment.

Agent 3: Customer Response Agent

Generate a customer-facing message based on the final workflow decision. The message must accurately communicate approval or rejection without claiming that a refund or replacement has already been completed.

Task 3 — Implement Business Rules

Implement the following demonstration return policy.

A request qualifies for automatic approval only when all conditions are satisfied:

1. Product amount is below ₹25,000.
2. Return request is within 30 days of delivery.
3. Customer invoice is available.
4. Item is unopened, or damaged with supporting photographs.
5. Customer has fewer than three previous returns in 90 days.
6. Risk Agent has not identified a HIGH-risk concern.

If any condition is not satisfied, the request must be escalated for human review.

Use deterministic Python checks for the business rules rather than relying entirely on AI-generated decisions.

Task 4 — Design the Agent Workflow

Using Microsoft Agent Framework, construct the following workflow:

```
Return Request → Eligibility Agent → Risk Agent → Approval Gateway → Customer Response Agent → Final Output
```

Use graph-based orchestration and clearly defined executor responsibilities.

Ensure request information passes correctly between agents.

Task 5 — Implement Human-in-the-Loop

For requests requiring human approval:

- Pause workflow execution using the appropriate Microsoft Agent Framework human-interaction mechanism.
- Display the request ID, item, amount, eligibility issues, and risk assessment.
- Ask the manager to enter `approve` or `reject`.
- Validate the entered response.
- Resume the paused workflow with the manager's decision.
- Preserve the request context throughout the workflow.

The AI must not override a human rejection.

Task 6 — Generate Final Output

For every request, generate a structured response containing:

- Request ID and order ID
- Product and requested amount
- Eligibility findings
- Risk assessment
- Approval route: Automatic or Human Review
- Final decision: AUTO_APPROVED, MANAGER_APPROVED, or MANAGER_REJECTED
- Customer communication draft

This is a simulation. Do not process actual payments, send messages, or initiate product replacements.

Task 7 — Test the Workflow

Execute the application against all 10 records.

Demonstrate:

- At least two automatic approvals.
- A request paused for human review and then approved.
- A different request paused for human review and then rejected.
- Correct workflow resumption.
- Proper handling of invalid approval input.
- Proper handling of missing API configuration.

Task 8 — Checkin to GIT

Submit the following:

1. `returns_workflow.py` — complete application.
2. `returns.json` — supplied input data.
3. `requirements.txt` — Python dependencies.
4. `.env.example` — configuration template without real credentials.
5. `README.md` — workflow explanation and execution steps.
6. Screenshots or console output demonstrating automatic approval, human approval, and human rejection.